

# When attention becomes a memory

## From token history to synaptic state

Modern language models seem to "look back" whenever they answer a question about earlier text. In a conventional Transformer, that intuition is concrete: each layer keeps key and value vectors for previous tokens, and a new query compares itself with those stored keys. The cache is effective and easy to reason about, but it grows with the sequence.

SYNAPSE MEMORY LAB / RESEARCH NOTE 01 / SEPTEMBER 2026

There is another route. For an important class of attention mechanisms, the past can be folded into a matrix whose shape never changes. New information is written by an outer product; retrieval is a matrix read. The Dragon Hatchling, or BDH, gives this familiar linear-algebra construction a provocative interpretation: the matrix is an evolving field of synaptic associations.

That connection is the heart of Synapse Memory Lab. It is also easy to overstate. The recurrence is exact for the linear kernel being evaluated, not for arbitrary softmax attention. "Synaptic" is a model interpretation, not proof that a neural network is a biological brain. And a fixed-size state avoids a growing cache by compressing history, which necessarily creates trade-offs.

## Start with the transparent version

Suppose each earlier position  $\tau$  produced a key  $k_\tau$  and value  $v_\tau$ , and the current position produces a query  $q_t$ . A simple strict-causal linear-attention read is

$$y_t = \sum_{\tau < t} (q_t^T k_\tau) v_\tau$$

The inner product says how strongly the current query matches an earlier key. Multiplying that scalar by the earlier value produces a contribution to the output. Summing the contributions retrieves a mixture of relevant values.

An explicit implementation is an excellent oracle: retain every pair  $(k_\tau, v_\tau)$ , loop over the history, and add its contribution. It also exposes a useful diagnostic—exactly how much each position contributed. But the retained arrays grow with time.

Now use associativity:

$$y_t = \sum_{\tau < t} q_t^T (k_\tau v_\tau^T) = q_t^T (\sum_{\tau < t} k_\tau v_\tau^T)$$

Define the parenthesized sum as  $S_{\{t-1\}}$ . Instead of remembering the pairs, maintain

$$S_t = S_{t-1} + k_t v_t^T$$

The result is the same for this kernel, but the implementation is recurrent. At time  $t$ , read  $q_t^T S_{t-1}$ , then write  $k_t v_t^T$ . That order matters. Writing first would let the current token attend to itself and would no longer match the strict-causal reference.

The 2020 paper [Transformers are RNNs](#) formalized this kernel-and-associativity route to recurrent linear attention. More recent systems make the state update selective. [Gated Linear Attention](#), for example, describes linear attention as a recurrent layer with a matrix-valued hidden state updated by outer products, while using data-dependent gates to control retention.

## Add forgetting, and see the trade-off

A scalar retention factor makes the mechanism easier to probe:

$$S_t = \gamma S_{t-1} + k_t v_t^T \quad 0 \leq \gamma \leq 1$$

Expanding it yields

$$S_{t-1} = \sum_{\tau < t} \gamma^{t-1-\tau} k_\tau v_\tau^T$$

The recurrent and explicit-history implementations are still equivalent if both use the same decay. But now the age of an association is visible in its coefficient. With  $\gamma < 1$ , an item  $a$  steps old is multiplied by  $\gamma^a$ . Forgetting is not a mysterious failure; it is built into the update.

Setting  $\gamma = 1$  avoids this deliberate attenuation, but does not give infinite capacity. Every association is superposed in the same finite matrix. Similar keys write into similar rows, so unrelated values can leak into a read. This is **interference**, the associative-memory analogue of collisions in a compressed data structure. Larger or better-separated representations, learned gating, normalization, and multi-head structure can help; none turns a fixed number of state variables into a lossless store of an unlimited token stream.

This is the honest bargain:

- A KV-style history preserves individually addressable token records, but its storage grows with the context.
- A recurrent matrix keeps fixed shape in sequence length, but individual records are no longer separately recoverable and can attenuate or interfere.

The asymptotic statement also needs care. "Constant memory" means constant **with respect to sequence length** for fixed key/value dimensions and a fixed number of layers and heads. A  $d_k \times d_v$  state may itself be large.

## Why call the update Hebbian?

The outer product changes a connection-like quantity using the simultaneous activities of two vectors. Coordinate-wise,

$$S_{ij} \leftarrow \gamma S_{ij} + k_i v_j$$

When key coordinate  $i$  and value coordinate  $j$  are both active, their association strengthens. This has the form of a local Hebbian or fast-weight write: activity changes a rapidly evolving connection state while the model's trained parameters remain fixed during inference.

That interpretation has a long lineage in machine learning. What BDH contributes is a more explicit attempt to align the tensor computation with a network of locally interacting neuron-like particles and synapses. In the tensor-friendly BDH-GPU equations, a recurrent matrix state is updated from current activity by an outer-product-like term and read by later activity. The paper's graph formulation interprets the corresponding state as evolving synaptic strengths.

The [BDH paper](#) additionally reports sparse positive activations, concept-sensitive individual synapses, modular/heavy-tailed graph structure, and language-model scaling comparisons with GPT-2-style Transformers. These are the authors' empirical findings. They should not be collapsed into the algebraic identity above: the identity can be checked line by line, while claims about learned representations and biological plausibility depend on experiments, modeling assumptions, and independent replication.

## What BDH-CQ adds-and what we can safely say

[BDH-CQ](#) applies this architecture family to in-context learning and recurrent latent reasoning. At a high level, demonstrations presented during inference update a recurrent contextual state. The model then performs iterative computation in a high-dimensional latent space rather than verbalizing every intermediate step as chain-of-thought text.

The authors evaluate a 150-million-parameter configuration on ARC-AGI-1 and report 29.5% pass@2 with a computed inference cost of \$0.0007 per task. Those numbers are notable, but their evidence status matters: they are results in a very recent 2026 preprint, not a mature consensus or a guarantee for other workloads. The public paper supports describing recurrent contextual memory and latent iteration. It does not license speculation about undisclosed or proprietary mechanics.

BDH-CQ's role in this conceptual map is therefore broader than the small recurrence demo. BDH gives the state a synaptic interpretation; BDH-CQ explores whether such recurrent memory can absorb demonstrations and support repeated latent computation. The lab teaches the common mechanical intuition without pretending to reproduce the complete trained model.

## How to evaluate the mechanism responsibly

A useful educational implementation should separate four kinds of evidence.

- 1. Algebraic guarantees.** For the same query, keys, values, decay, and read-before-write convention, the explicit sum and recurrent state should agree numerically. Unit tests can establish this within a floating-point tolerance.
- 2. Structural guarantees.** The recurrent state's array length should remain `d_kd_v` as more tokens arrive. Tests can also confirm reset behavior, dimension checks, deterministic seeds, and the exclusion of the current token.
- 3. Toy empirical behavior.** Associative-recall experiments can vary load, key similarity, state width, and decay. Falling recall demonstrates compression pressure in that toy setup. It is not a calibrated capacity claim about trained BDH.
- 4. Paper-reported behavior.** BDH sparsity, synapse interpretability, scaling comparisons, and BDH-CQ benchmark results belong in a cited results section, labeled as author-reported until independently reproduced.

This separation prevents two common mistakes. First, a correlation between softmax weights and linear-state retrieval in one visualization does not prove equivalence to softmax. Second, an evocative biological interpretation does not turn a simplified browser simulation into the research model.

## Strengths worth keeping

The recurrent view is valuable even without biological language. It supports streaming computation, bounded state size, and efficient autoregressive reads. It turns context handling into explicit state dynamics, making retention, decay, and writes inspectable. The outer-product rule also connects attention, fast weights, associative memory, and test-time adaptation under one compact mathematical picture. Recent work such as [test-time training with expressive hidden states](#) explores a neighboring idea: make the recurrent state itself a richer learning system.

BDH's synaptic framing can also be pedagogically useful. "Write an association now; let future activity retrieve it" is often easier to reason about than a large attention tensor. Sparse positive activity may make individual state changes easier to visualize, although interpretability in a toy cell does not establish interpretability at production scale.

## Limitations that define the research agenda

Fixed-state memory faces attenuation, interference, finite precision, and a loss of token-level provenance. Training must learn representations and dynamics that use the available state well. Hardware efficiency is not automatic: matrix states and their training gradients can be expensive even when inference memory does not grow with context. Comparisons must control parameter count, data, compute, context length, and quality-not just asymptotic notation.

There is also a maturity gap. Recurrent linear attention is an established technique with several peer-reviewed and openly studied ancestors. Gating, fast-weight memories, and test-time-learning states remain fast-moving research areas. BDH, introduced in 2025, has public code but a short replication history. BDH-CQ, posted in August 2026, is newer still. The right stance is neither dismissal nor hype: reproduce the equations, test the claimed invariants, treat benchmark results as provisional evidence, and identify which conclusions survive across datasets and independent implementations.

## The takeaway

Attention does not always require retaining an explicit list of past tokens. For causal linear attention, the list and a recurrent outer-product state are two evaluation strategies for the same kernel. The recurrent form replaces history growth with compression into a fixed-shape matrix. BDH interprets that matrix as synaptic state; BDH-CQ investigates recurrent contextual memory and latent reasoning on top of the architectural family.

The fixed shape is the attraction-and the constraint. It makes streaming memory possible, but forces information to share a finite substrate. Once that trade-off is visible, "attention as synaptic plasticity" becomes more than a metaphor: it becomes a precise, testable computational story whose boundaries are as important as its promise.

## Primary reading

- Kosowski, Uznański, Chorowski, Stamirowska, and Bartoszkiewicz (2025), [The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain](#).
- Engdahl et al. (2026), [BDH-CQ: In-Context Learning with Recurrent Latent Reasoning](#).
- Yang, Wang, Shen, Panda, and Zhang (2023; revised 2024), [Gated Linear Attention Transformers with Hardware-Efficient Training](#).
- Sun et al. (2024), [Learning to \(Learn at Test Time\): RNNs with Expressive Hidden States](#).
- Katharopoulos, Vyas, Pappas, and Fleuret (2020), [Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention](#).